MEAN STACK

(LPEIT-109)

PRACTICAL FILE

GURU NANAK DEV ENGINEERING COLLEGE

SUBMITTED IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE OF

BACHELORS OF TECHNOLOGY

(INFORMATION TECHNOLOGY)

SUBMITTED BY:

SHARNJEET SINGH (2303059)

DEPARTMENT OF INFORMATION TECHNOLOGY

GURU NANAK DEV ENGINEERING COLLEGE LUDHIANA

(An Autonomous College Under UGC ACT)

1. Practical No. 1: Installing and Setting Up Node.js, npm, and Express

1.1 Aim of the Practical

To install, setup and configure Node.js, npm (Node Package Manager), and Express framework for building web applications.

1.2 Introduction to Node.js

Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside a web browser. It uses Chrome's V8 JavaScript engine and enables developers to build scalable server-side applications using JavaScript.

1.3 Introduction to npm

npm (Node Package Manager) is the default package manager for Node.js. It allows developers to install, share, and manage dependencies (libraries and tools) in their projects. npm comes bundled with Node.js installation.

1.4 Introduction to Express

Express is a minimal and flexible Node.js web application framework that provides robust features for building web and mobile applications. It simplifies server creation, routing, and middleware integration.

1.5 Installation Steps

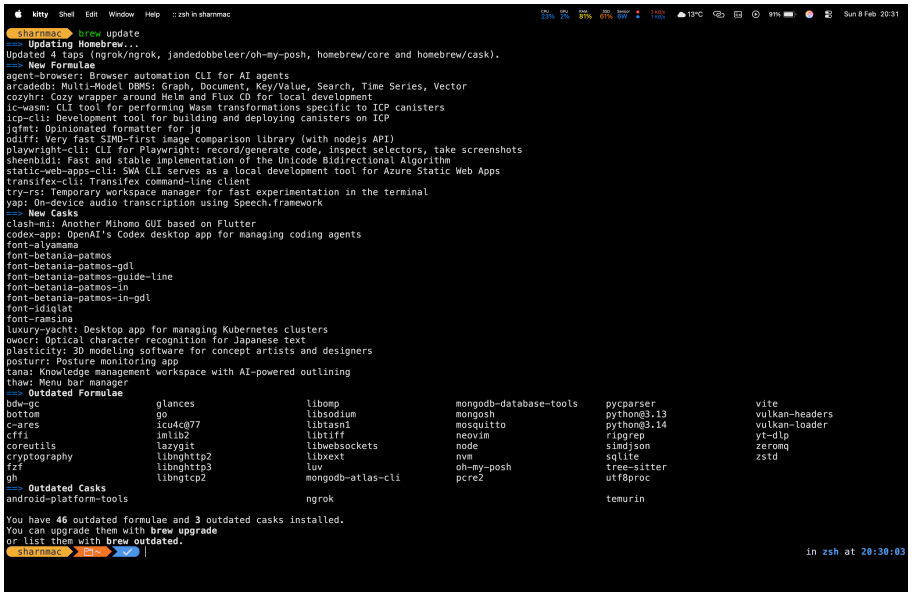
1.5.1 Install Node.js and npm on macOS

There are multiple ways to install Node.js on macOS. We'll use Homebrew, the most popular package manager for macOS:

Step 1: Update Homebrew

```
brew update
```

Output:

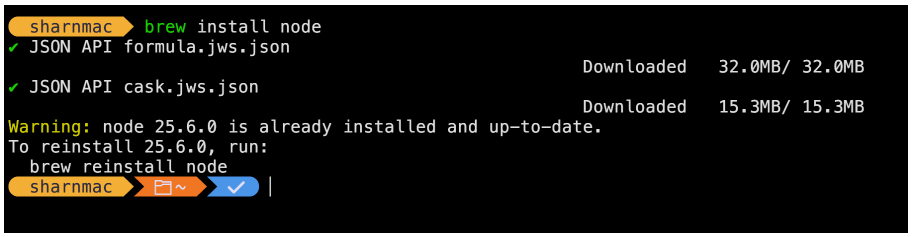


Step 2: Install Node.js using Homebrew

Node.js installation includes npm automatically:

```
brew install node
```

Output:



Alternative: Download from Official Website

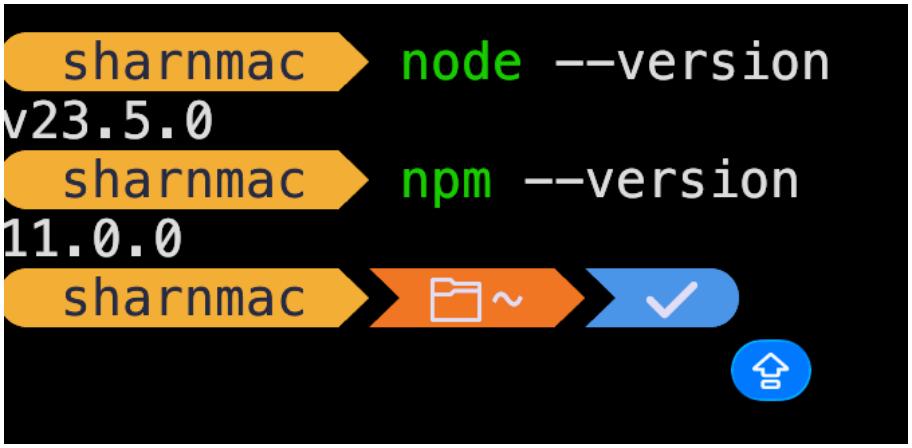
You can also download the macOS installer (.pkg file) from `https://nodejs.org/` and install it manually.

Step 3: Verify installation

Check Node.js and npm versions:

```
node --version
npm --version
```

Output:

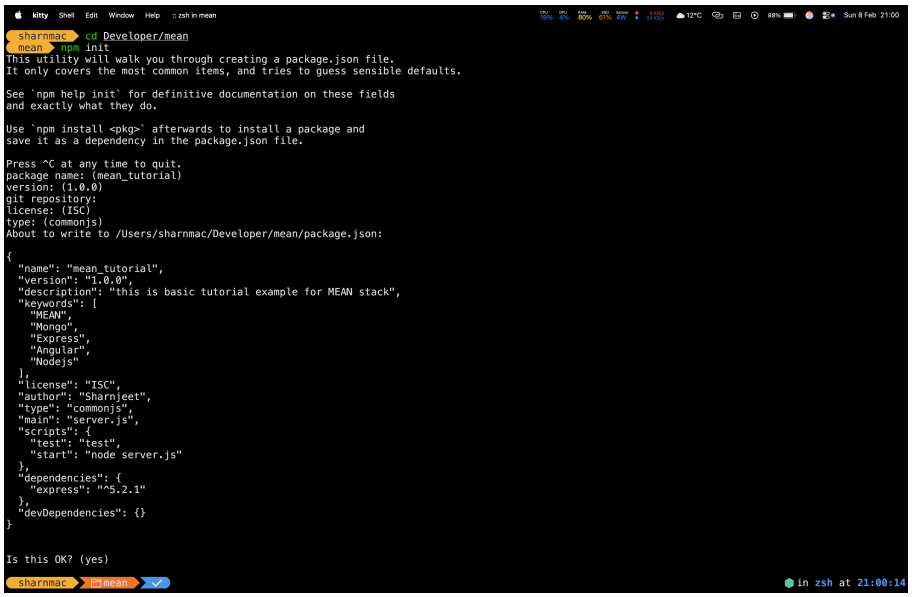


1.5.2 Initialize a New Node.js Project

Create a new directory called `mean` and initialize npm:

```
mkdir mean
cd mean
npm init
```

Output:

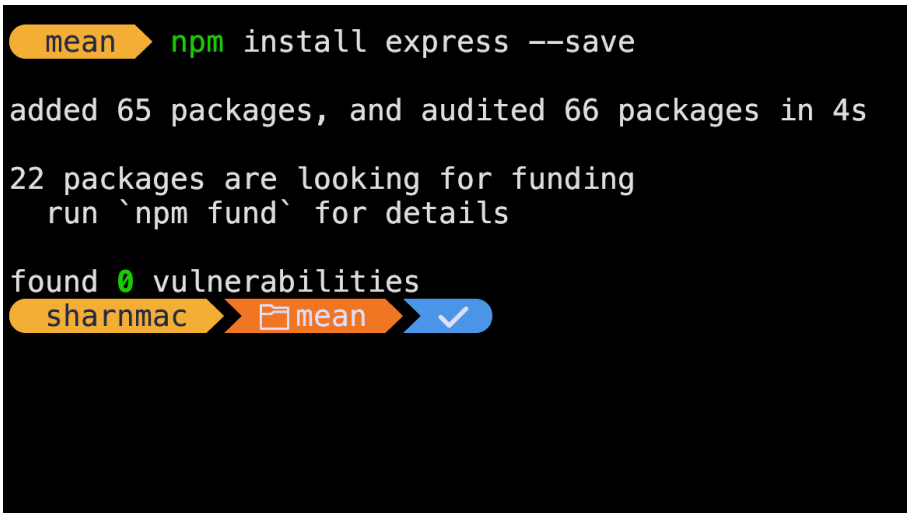


1.5.3 Install Express Framework

Install Express as a project dependency:

```
npm install express
```

Output:



This command downloads Express and adds it to the `node_modules` folder and `package.json` dependencies.

1.6 Project File Structure in mean Folder

After installation, the `mean` directory contains:

```
mean/
|-- node_modules/        # All installed packages
|-- package.json         # Project metadata and dependencies
|-- package-lock.json    # Locked versions of dependencies
```

1.7 Understanding package.json

The `package.json` file contains project configuration:

```
{
  "name": "mean",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

1.8 Key Concepts

- **dependencies**: Packages required for production
- **devDependencies**: Packages needed only for development
- **node_modules**: Folder containing all installed packages
- **package-lock.json**: Ensures consistent installations across environments

2. Practical No. 2: Creating Express Project and Building Server

2.1 Aim of the Practical

To create an Express project and build a basic server using Express and Node.js with routing.

2.2 Introduction

Express simplifies web server creation by providing routing capabilities and middleware support. In this practical, we will create a basic Express server that handles different routes and sends responses directly from the server.

2.3 Complete Project Structure in mean Folder

The `mean` folder contains the following files and directories:

```
mean/                    # Main project directory
|-- node_modules/        # Installed packages (auto-generated)
|-- package.json         # Project configuration
|-- package-lock.json    # Dependency lock file
|-- server.js            # Main server file
```

2.4 File Descriptions

2.4.1 server.js - Main Server File

The `server.js` file is the entry point of the application. It configures Express, defines routes, and starts the server.

Key Components:

- Import Express module
- Create Express application instance
- Define routes for different endpoints
- Set up error handling
- Start server on specified port

server.js Code:

```
const express = require('express');
const app = express();

const port = 3000;

app.get('/', (req, res) => res.send("Hello"));

app.listen(port, () => console.log("hello from server"));
```

2.5 Configuration in package.json

The `package.json` file with configuration:

```
{
  "name": "mean_tutorial",
  "version": "1.0.0",
  "description": "this is basic tutorial example for MEAN stack",
  "keywords": [
    "MEAN",
    "Mongo",
    "Express",
    "Angular",
    "Nodejs"
  ],
  "license": "ISC",
  "author": "Sharnjeet",
  "type": "commonjs",
  "main": "server.js",
  "scripts": {
    "test": "test"
  },
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

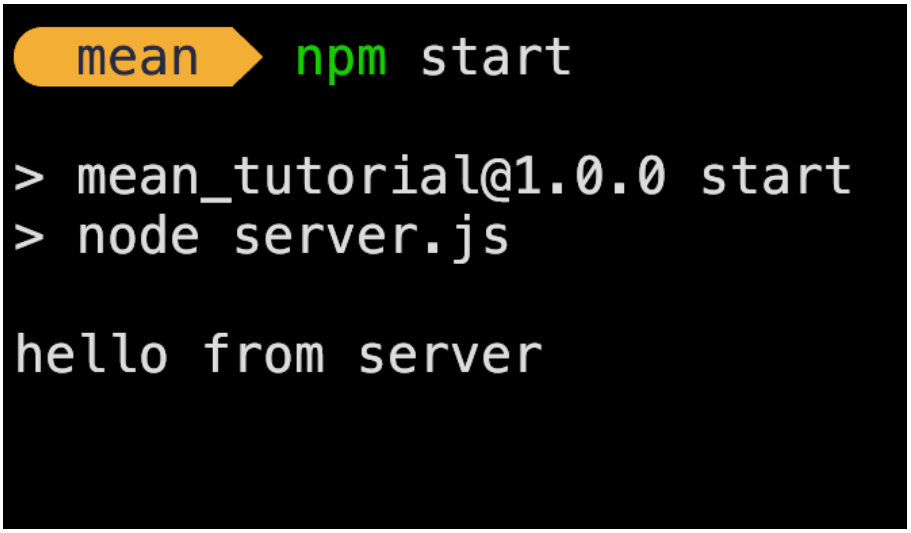
2.6 Running the Server

2.6.1 Production Mode

Start the server normally:

```
npm start
```

Output:

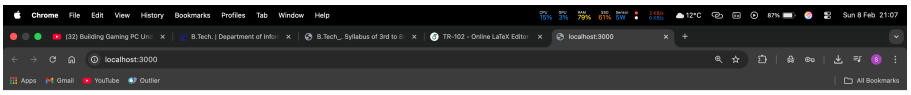


2.7 Accessing the Server

Once the server is running, open a web browser and navigate to:

- Home page: `http://localhost:3000/`
- Invalid route: `http://localhost:3000/invalid` (shows 404 error)

Browser Output:



Hello

2.8 Key Concepts

2.8.1 Express Application

`express()` creates an Express application instance that handles HTTP requests and responses.

2.8.2 Routing

Express uses `app.get()` to define routes:

- First parameter: URL path (e.g., '/', '/about', '/contact')
- Second parameter: Callback function with request (req) and response (res) objects
- `res.send()`: Sends HTML content as response

2.8.3 Port Configuration

`process.env.PORT || 3000` allows the server to use environment variable PORT or default to 3000.

2.8.4 Error Handling

The 404 handler uses `app.use()` to catch all undefined routes and sends an appropriate error message.

2.8.5 Server Listening

`app.listen()` starts the server on specified port and executes callback function when server starts successfully.

2.9 Advantages of This Setup

- **Simple Structure**: Minimal files for quick setup and learning
- **Easy to Understand**: All server logic in one file
- **Scalable**: Can easily add more routes and functionality
- **Fast Development**: nodemon enables quick testing during development

3. Practical No. 3: Working with Bootstrap for Responsive Layouts

3.1 Aim of the Practical

To import and implement Bootstrap framework for creating quick, responsive layouts in web applications.

3.2 Introduction to Bootstrap

Bootstrap is a free, open-source CSS framework that provides pre-built components and responsive grid system for building modern, mobile-first websites. It includes HTML, CSS, and JavaScript components for typography, forms, buttons, navigation, and other interface elements.

3.3 Key Features of Bootstrap

- **Responsive Grid System**: 12-column layout that adapts to different screen sizes
- **Pre-styled Components**: Buttons, forms, cards, modals, and more
- **Mobile-First Approach**: Optimized for mobile devices by default
- **Cross-Browser Compatibility**: Works consistently across modern browsers
- **Customizable**: Easy to override default styles

3.4 Methods to Import Bootstrap

3.4.1 Method 1: CDN (Content Delivery Network)

Add Bootstrap via CDN links in your HTML file. This is the quickest method and doesn't require installation.

HTML with Bootstrap CDN:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
       scale=1.0">
    <title>Bootstrap Example</title>

    <!-- Bootstrap CSS -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/
       css/bootstrap.min.css"
          rel="stylesheet">
</head>
<body>
    <div class="container">
        <h1 class="text-center mt-5">Welcome to Bootstrap</h1>
```

```
        <p class="lead">This is a responsive layout using
           Bootstrap.</p>
    </div>

    <!-- Bootstrap JS Bundle -->
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist
       /js/bootstrap.bundle.min.js">
    </script>
</body>
</html>
```

3.4.2 Method 2: npm Installation

Install Bootstrap as a project dependency using npm:

```
npm install bootstrap
```

Then import Bootstrap in your JavaScript or CSS files:

```
// In your main JavaScript file
import 'bootstrap/dist/css/bootstrap.min.css';
import 'bootstrap/dist/js/bootstrap.bundle.min.js';
```

3.5 Creating a Responsive Layout

3.5.1 Bootstrap Grid System

The grid system uses containers, rows, and columns to layout content:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,␣initial-
       scale=1.0">
    <title>Responsive Grid Layout</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/
       css/bootstrap.min.css"
          rel="stylesheet">
</head>
<body>
    <div class="container␣mt-5">
        <div class="row">
            <div class="col-md-4␣col-sm-6␣col-12␣mb-3">
                <div class="card">
                    <div class="card-body">
                        <h5 class="card-title">Column 1</h5>
                        <p class="card-text">This is a responsive
                           column.</p>
                    </div>
                </div>
            </div>
```

```
            <div class="col-md-4 col-sm-6 col-12 mb-3">
                <div class="card">
                    <div class="card-body">
                        <h5 class="card-title">Column 2</h5>
                        <p class="card-text">Adapts to screen size
                            .</p>
                    </div>
                </div>
            </div>
            <div class="col-md-4 col-sm-12 col-12 mb-3">
                <div class="card">
                    <div class="card-body">
                        <h5 class="card-title">Column 3</h5>
                        <p class="card-text">Mobile-first design.<
                            /p>
                    </div>
                </div>
            </div>
        </div>
    </div>

    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist
        /js/bootstrap.bundle.min.js">
    </script>
</body>
</html>
```

3.6 Common Bootstrap Components

3.6.1 Navigation Bar

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
        <a class="navbar-brand" href="#">MyApp</a>
        <button class="navbar-toggler" type="button" data-bs-
            toggle="collapse"
                data-bs-target="#navbarNav">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav">
                <li class="nav-item">
                    <a class="nav-link active" href="#">Home</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="#">About</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="#">Contact</a>
                </li>
```

```
            </ul>
        </div>
    </div>
</nav>
```

3.6.2 Forms

```
<div class="container␣mt-5">
    <form>
        <div class="mb-3">
            <label for="email" class="form-label">Email address</
               label>
            <input type="email" class="form-control" id="email"
                   placeholder="name@example.com">
        </div>
        <div class="mb-3">
            <label for="password" class="form-label">Password</
               label>
            <input type="password" class="form-control" id="
               password">
        </div>
        <button type="submit" class="btn␣btn-primary">Submit</
           button>
    </form>
</div>
```

3.6.3 Buttons

```
<button type="button" class="btn␣btn-primary">Primary</button>
<button type="button" class="btn␣btn-secondary">Secondary</button>
<button type="button" class="btn␣btn-success">Success</button>
<button type="button" class="btn␣btn-danger">Danger</button>
<button type="button" class="btn␣btn-warning">Warning</button>
<button type="button" class="btn␣btn-info">Info</button>
```

3.7 Responsive Breakpoints

Bootstrap uses the following breakpoints for responsive design:

- **xs** (Extra small): $<$ 576px (default, no prefix needed)
- **sm** (Small): $>=$ 576px
- **md** (Medium): $>=$ 768px
- **lg** (Large): $>=$ 992px
- **xl** (Extra large): $>=$ 1200px
- **xxl** (Extra extra large): $>=$ 1400px

3.8 Integrating Bootstrap with Express

Create a public folder to serve static files and add Bootstrap:

Updated server.js:

```
const express = require('express');
const path = require('path');
const app = express();

const port = 3000;

// Serve static files from 'public' directory
app.use(express.static(path.join(__dirname, 'public')));

app.get('/', (req, res) => {
    res.sendFile(path.join(__dirname, 'public', 'index.html'));
});

app.listen(port, () => {
    console.log(`Server running on http://localhost:${port}`);
});
```

3.9 Advantages of Using Bootstrap

- **Rapid Development**: Pre-built components speed up development
- **Consistency**: Uniform design across the application
- **Responsive by Default**: Automatically adapts to different screen sizes
- **Large Community**: Extensive documentation and community support
- **Customizable**: Easy to modify with custom CSS

4. Practical No. 4: MongoDB Installation and CRUD Operations

4.1 Aim of the Practical

To install MongoDB and perform CRUD (Create, Read, Update, Delete) operations on documents and databases.

4.2 Introduction to MongoDB

MongoDB is a NoSQL, document-oriented database that stores data in flexible, JSON-like documents called BSON (Binary JSON). Unlike traditional relational databases, MongoDB doesn't require a predefined schema, making it ideal for applications with evolving data requirements.

4.3 Key Features of MongoDB

- **Document-Oriented**: Stores data in JSON-like documents
- **Schema-less**: No rigid table structure required
- **Scalable**: Horizontal scaling through sharding
- **High Performance**: Fast read and write operations
- **Flexible Queries**: Rich query language with indexing support

4.4 MongoDB Installation on macOS

4.4.1 Step 1: Install MongoDB using Homebrew

```
# Tap the MongoDB Homebrew repository
brew tap mongodb/brew

# Install MongoDB Community Edition
brew install mongodb-community
```

4.4.2 Step 2: Start MongoDB Service

```
# Start MongoDB as a service
brew services start mongodb-community

# Verify MongoDB is running
brew services list
```

4.4.3 Step 3: Access MongoDB Shell

```
# Open MongoDB shell
mongosh
```

4.5 MongoDB Basic Concepts

- **Database**: Container for collections (similar to a database in SQL)
- **Collection**: Group of documents (similar to a table in SQL)
- **Document**: A record in a collection (similar to a row in SQL)
- **Field**: A key-value pair in a document (similar to a column in SQL)

4.6 CRUD Operations in MongoDB Shell

4.6.1 Create Operations

1. Create/Switch to a Database:

```
use studentDB
```

2. Insert a Single Document:

```
db.students.insertOne({
    name: "Sharnjeet Singh",
    rollNo: 2303059,
    branch: "Information Technology",
    semester: 6,
    cgpa: 8.5
})
```

3. Insert Multiple Documents:

```
db.students.insertMany([
    {
        name: "Rajesh Kumar",
        rollNo: 2303060,
        branch: "Computer Science",
        semester: 6,
        cgpa: 8.2
    },
    {
        name: "Priya Sharma",
        rollNo: 2303061,
        branch: "Information Technology",
        semester: 6,
        cgpa: 9.1
    },
    {
        name: "Amit Verma",
        rollNo: 2303062,
        branch: "Electronics",
        semester: 5,
        cgpa: 7.8
    }
])
```

4.6.2 Read Operations

1. Find All Documents:

```
db.students.find()
```

2. Find with Pretty Format:

```
db.students.find().pretty()
```

3. Find Specific Documents (Query):

```
// Find students from IT branch
db.students.find({ branch: "Information Technology" })

// Find students with CGPA greater than 8.0
db.students.find({ cgpa: { $gt: 8.0 } })

// Find a specific student by roll number
db.students.findOne({ rollNo: 2303059 })
```

4. Find with Projection (Select Specific Fields):

```
// Show only name and cgpa, hide _id
db.students.find(
    {},
    { name: 1, cgpa: 1, _id: 0 }
)
```

5. Find with Sorting:

```
// Sort by CGPA in descending order
db.students.find().sort({ cgpa: -1 })

// Sort by name in ascending order
db.students.find().sort({ name: 1 })
```

6. Find with Limit:

```
// Get only first 2 students
db.students.find().limit(2)
```

4.6.3 Update Operations

1. Update a Single Document:

```
// Update CGPA of a specific student
db.students.updateOne(
    { rollNo: 2303059 },
    { $set: { cgpa: 8.7 } }
)
```

2. Update Multiple Documents:

```
// Add email field to all IT students
db.students.updateMany(
    { branch: "Information Technology" },
```

```
    { $set: { email: "student@gndec.ac.in" } }
)
```

3. Increment a Field Value:

```
// Increment semester by 1
db.students.updateOne(
    { rollNo: 2303059 },
    { $inc: { semester: 1 } }
)
```

4. Add a New Field:

```
// Add phone number field
db.students.updateOne(
    { rollNo: 2303059 },
    { $set: { phone: "9876543210" } }
)
```

5. Remove a Field:

```
// Remove email field
db.students.updateOne(
    { rollNo: 2303059 },
    { $unset: { email: "" } }
)
```

6. Replace Entire Document:

```
db.students.replaceOne(
    { rollNo: 2303059 },
    {
        name: "Sharnjeet Singh",
        rollNo: 2303059,
        branch: "IT",
        semester: 7,
        cgpa: 8.9,
        status: "active"
    }
)
```

4.6.4 Delete Operations

1. Delete a Single Document:

```
// Delete student with specific roll number
db.students.deleteOne({ rollNo: 2303062 })
```

2. Delete Multiple Documents:

```
// Delete all students with CGPA less than 8.0
db.students.deleteMany({ cgpa: { $lt: 8.0 } })
```

3. Delete All Documents in Collection:

```
// Delete all documents (but keep the collection)
db.students.deleteMany({})
```

4.7 Additional MongoDB Operations

4.7.1 Database Operations

```
// Show all databases
show dbs

// Show current database
db

// Drop current database
db.dropDatabase()
```

4.7.2 Collection Operations

```
// Show all collections in current database
show collections

// Create a new collection explicitly
db.createCollection("courses")

// Drop a collection
db.students.drop()

// Count documents in collection
db.students.countDocuments()

// Count with filter
db.students.countDocuments({ branch: "Information Technology" })
```

4.8 Query Operators in MongoDB

4.8.1 Comparison Operators

```
// $eq - Equal to
db.students.find({ semester: { $eq: 6 } })

// $ne - Not equal to
db.students.find({ branch: { $ne: "Electronics" } })

// $gt - Greater than
db.students.find({ cgpa: { $gt: 8.5 } })

// $gte - Greater than or equal to
db.students.find({ cgpa: { $gte: 8.5 } })

// $lt - Less than
db.students.find({ semester: { $lt: 6 } })
```

```
// $lte - Less than or equal to
db.students.find({ semester: { $lte: 6 } })

// $in - In array
db.students.find({ branch: { $in: ["IT", "CSE"] } })

// $nin - Not in array
db.students.find({ semester: { $nin: [1, 2, 3] } })
```

4.8.2 Logical Operators

```
// $and - Logical AND
db.students.find({
    $and: [
        { branch: "Information Technology" },
        { cgpa: { $gt: 8.0 } }
    ]
})

// $or - Logical OR
db.students.find({
    $or: [
        { branch: "IT" },
        { branch: "CSE" }
    ]
})

// $not - Logical NOT
db.students.find({
    cgpa: { $not: { $lt: 8.0 } }
})

// $nor - Logical NOR
db.students.find({
    $nor: [
        { branch: "Electronics" },
        { semester: 5 }
    ]
})
```

4.9 Connecting MongoDB with Node.js

4.9.1 Install MongoDB Driver

```
npm install mongodb
```

4.9.2 Basic Connection Example

```
const { MongoClient } = require('mongodb');

// Connection URL
const url = 'mongodb://localhost:27017';
const client = new MongoClient(url);

// Database Name
const dbName = 'studentDB';

async function main() {
    // Connect to MongoDB
    await client.connect();
    console.log('Connected successfully to MongoDB');

    const db = client.db(dbName);
    const collection = db.collection('students');

    // Insert a document
    const insertResult = await collection.insertOne({
        name: "John Doe",
        rollNo: 2303063,
        branch: "IT",
        cgpa: 8.3
    });
    console.log('Inserted document:', insertResult);

    // Find documents
    const findResult = await collection.find({}).toArray();
    console.log('Found documents:', findResult);

    // Close connection
    await client.close();
}

main().catch(console.error);
```

4.10 Advantages of MongoDB

- **Flexible Schema**: Easy to modify data structure without migrations
- **Scalability**: Horizontal scaling through sharding
- **Performance**: Fast read/write operations with indexing
- **JSON-like Documents**: Natural fit for JavaScript applications
- **Rich Query Language**: Powerful querying and aggregation capabilities
- **High Availability**: Replication and automatic failover

4.11 Best Practices

- Always use meaningful collection and field names
- Create indexes on frequently queried fields
- Use projection to retrieve only necessary fields
- Validate data before insertion
- Handle errors properly in production applications
- Use connection pooling for better performance
- Close database connections when done

5. Practical No. 5: Building a Data Model with MongoDB and Mongoose

5.1 Aim of the Practical

To build a data model using MongoDB and Mongoose ODM (Object Data Modeling) library in a Node.js application.

5.2 Introduction to Mongoose

Mongoose is an ODM (Object Data Modeling) library for MongoDB and Node.js. It provides a schema-based solution to model application data, offering built-in type casting, validation, query building, and business logic hooks. Mongoose acts as a bridge between MongoDB's flexible document model and the structured data requirements of an application.

5.3 Key Features of Mongoose

- **Schema Definition**: Define the structure and data types of documents
- **Validation**: Built-in and custom validators for data integrity
- **Middleware (Hooks)**: Pre and post hooks for document lifecycle events
- **Query Helpers**: Chainable query API for complex queries
- **Population**: Reference documents across collections (like SQL joins)
- **Plugins**: Reusable logic that can be attached to schemas

5.4 Installation

```
npm install mongoose
```

5.5 Project Structure

```
mean/
|-- node_modules/
|-- models/
|   |-- User.js          # User data model
|   |-- Itinerary.js     # Itinerary data model
|-- server.js
|-- package.json
```

5.6 Defining Mongoose Schemas and Models

5.6.1 User Model (models/User.js)

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
      minlength: [2, 'Name must be at least 2 characters'],
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email'],
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minlength: [6, 'Password must be at least 6 characters'],
    },
    role: {
      type: String,
      enum: ['user', 'admin', 'superadmin'],
      default: 'user',
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model('User', userSchema);
```

5.6.2 Itinerary Model (models/Itinerary.js)

```
const mongoose = require('mongoose');

const itinerarySchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, 'Title is required'],
      trim: true,
    },
    destination: {
      type: String,
```

```
      required: [true, 'Destination is required'],
    },
    duration: {
      type: Number,
      required: true,
      min: [1, 'Duration must be at least 1 day'],
    },
    budget: {
      type: Number,
      required: true,
      min: [0, 'Budget cannot be negative'],
    },
    description: { type: String, trim: true },
    createdBy: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
    },
    status: {
      type: String,
      enum: ['active', 'inactive', 'draft'],
      default: 'active',
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model('Itinerary', itinerarySchema);
```

5.7 Using the Models

```
const User = require('./models/User');
const Itinerary = require('./models/Itinerary');

// Create a new user
const newUser = new User({
  name: 'Sharnjeet Singh',
  email: 'sharnjeet@example.com',
  password: 'secret123',
  role: 'user',
});

await newUser.save();
console.log('User saved:', newUser);

// Create an itinerary linked to the user
const trip = await Itinerary.create({
  title: 'Goa Beach Escape',
  destination: 'Goa, India',
  duration: 5,
```

```
  budget: 15000,
  description: 'A relaxing beach holiday.',
  createdBy: newUser._id,
});

console.log('Itinerary created:', trip);
```

5.8 Schema Data Types

- **String**: Text values
- **Number**: Integer or floating-point numbers
- **Boolean**: `true` or `false`
- **Date**: Date and time values
- **Array**: List of values or sub-documents
- **ObjectId**: Reference to another document (`mongoose.Schema.Types.ObjectId`)
- **Mixed**: Any type (`mongoose.Schema.Types.Mixed`)

5.9 Validation in Mongoose

```
const productSchema = new mongoose.Schema({
  name:  { type: String, required: true, maxlength: 100 },
  price: { type: Number, required: true, min: 0 },
  category: {
    type: String,
    enum: ['electronics', 'clothing', 'food'],
    required: true,
  },
  inStock: { type: Boolean, default: true },
  createdAt: { type: Date, default: Date.now },
});
```

5.10 Advantages of Using Mongoose

- **Structured Data**: Enforces schema on otherwise schema-less MongoDB
- **Validation**: Prevents invalid data from reaching the database
- **Readable Code**: Models make the codebase self-documenting
- **Middleware**: Automate tasks like hashing passwords before saving
- **Population**: Easily join related documents across collections

6. Practical No. 6: Connecting Express Application to MongoDB using Mongoose

6.1 Aim of the Practical

To connect an Express application to a MongoDB database using Mongoose and build a fully functional REST API with CRUD operations.

6.2 Introduction

Connecting Express with MongoDB via Mongoose allows us to build a complete backend API. Express handles HTTP routing and middleware, while Mongoose manages database interactions through models. Together they form the **M** (MongoDB) and **E** (Express) layers of the MEAN stack.

6.3 Installation

```
npm install express mongoose dotenv
```

6.4 Project Structure

```
mean/
|-- config/
|   |-- db.js             # Database connection
|-- models/
|   |-- Itinerary.js      # Mongoose model
|-- routes/
|   |-- itineraryRoutes.js  # API routes
|-- server.js             # Entry point
|-- .env                  # Environment variables
|-- package.json
```

6.5 Environment Variables (.env)

```
MONGO_URI=mongodb://localhost:27017/meanDB
PORT=5000
```

6.6 Database Connection (config/db.js)

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
```

```
    console.error(`Error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

6.7 Express Server (server.js)

```
require('dotenv').config();
const express = require('express');
const connectDB = require('./config/db');
const itineraryRoutes = require('./routes/itineraryRoutes');

const app = express();

// Connect to MongoDB
connectDB();

// Middleware
app.use(express.json());

// Routes
app.use('/api/itineraries', itineraryRoutes);

// Health check
app.get('/', (req, res) => res.json({ message: 'API is running' })
   );

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT
   }`));
```

6.8 REST API Routes (routes/itineraryRoutes.js)

```
const express = require('express');
const router = express.Router();
const Itinerary = require('../models/Itinerary');

// GET all itineraries
router.get('/', async (req, res) => {
  try {
    const itineraries = await Itinerary.find();
    res.json({ success: true, data: itineraries });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message })
       ;
  }
});
```

```
// GET single itinerary by ID
router.get('/:id', async (req, res) => {
  try {
    const item = await Itinerary.findById(req.params.id);
    if (!item) return res.status(404).json({ message: 'Not found'
       });
    res.json({ success: true, data: item });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message })
       ;
  }
});

// POST create new itinerary
router.post('/', async (req, res) => {
  try {
    const item = await Itinerary.create(req.body);
    res.status(201).json({ success: true, data: item });
  } catch (err) {
    res.status(400).json({ success: false, message: err.message })
       ;
  }
});

// PUT update itinerary
router.put('/:id', async (req, res) => {
  try {
    const item = await Itinerary.findByIdAndUpdate(
      req.params.id, req.body, { new: true, runValidators: true }
    );
    if (!item) return res.status(404).json({ message: 'Not found'
       });
    res.json({ success: true, data: item });
  } catch (err) {
    res.status(400).json({ success: false, message: err.message })
       ;
  }
});

// DELETE itinerary
router.delete('/:id', async (req, res) => {
  try {
    const item = await Itinerary.findByIdAndDelete(req.params.id);
    if (!item) return res.status(404).json({ message: 'Not found'
       });
    res.json({ success: true, message: 'Deleted successfully' });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message })
       ;
  }
```

```
});

module.exports = router;
```

6.9 Testing the API

Start the server and test endpoints using a browser or API client (e.g., Postman):

```
node server.js
# Output: MongoDB Connected: localhost
#         Server running on port 5000
```

- **GET** `http://localhost:5000/api/itineraries` — Fetch all records
- **POST** `http://localhost:5000/api/itineraries` — Create a record
- **PUT** `http://localhost:5000/api/itineraries/:id` — Update a record
- **DELETE** `http://localhost:5000/api/itineraries/:id` — Delete a record

6.10 Key Concepts

- **async/await**: Handles asynchronous MongoDB operations cleanly
- **try/catch**: Catches database errors and sends appropriate HTTP responses
- **dotenv**: Keeps sensitive credentials out of source code
- **process.exit(1)**: Terminates the app if the DB connection fails
- **findByIdAndUpdate**: Atomically finds and updates a document

6.11 Advantages

- **Separation of Concerns**: Config, models, and routes are in separate files
- **Reusable Models**: Mongoose models can be imported anywhere in the app
- **Error Handling**: Centralized error responses for all routes
- **Environment Config**: `.env` file keeps credentials secure

7. Practical No. 7: Creating an Angular Application and Working with Components

7.1 Aim of the Practical

To create an Angular application and understand how to build, organize, and communicate between Angular components.

7.2 Introduction to Angular

Angular is a TypeScript-based, open-source front-end framework developed and maintained by Google. It follows the component-based architecture where the UI is broken into small, reusable, self-contained pieces called **components**. Angular provides a complete solution including routing, forms, HTTP client, and dependency injection out of the box.

7.3 Key Concepts

- **Component**: The fundamental building block of an Angular UI
- **Template**: HTML view associated with a component
- **Decorator**: Metadata annotation (e.g., `@Component`) that configures a class
- **Module / Standalone**: Logical grouping of components and services
- **Data Binding**: Synchronization between component class and template
- **Directive**: Instructions that extend HTML (e.g., `*ngIf`, `*ngFor`)
- **Service**: Reusable business logic shared across components

7.4 Installation and Project Setup

7.4.1 Install Angular CLI

```
npm install -g @angular/cli
```

7.4.2 Create a New Angular Project

```
ng new frontend --standalone --routing --style=scss
cd frontend
```

7.4.3 Project Structure

```
frontend/
|-- src/
|   |-- app/
|   |   |-- app.ts                # Root component
|   |   |-- app.routes.ts         # Route definitions
|   |   |-- app.config.ts         # App configuration
|   |   |-- components/
|   |   |   |-- navbar/           # Navbar component
|   |   |   |-- home/             # Home component
|   |   |   |-- dashboard/        # Dashboard component
|   |   |-- services/
|   |       |-- api.service.ts    # HTTP service
|   |-- index.html
|   |-- styles.scss
|-- angular.json
|-- package.json
```

7.5 Anatomy of a Component

Every Angular component consists of three parts:

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-home',        // HTML tag used in templates
  standalone: true,            // No NgModule required (Angular
     17+)
  imports: [CommonModule],     // Dependencies for this component
  template: `
    <section>
      <h1>{{ title }}</h1>
      <p *ngIf="showMessage">Welcome to the MEAN Stack App!</p>
      <ul>
        <li *ngFor="let item of items">{{ item }}</li>
      </ul>
    </section>
  `,
})
export class HomeComponent {
  title = 'Travel Itinerary Planner';
  showMessage = true;
  items = ['Plan trips', 'Explore destinations', 'Manage
     itineraries'];
}
```

7.6 Data Binding Types

7.6.1 Interpolation (One-way: Class to Template)

```
<h1>{{ title }}</h1>
<p>Welcome, {{ user.name }}!</p>
```

7.6.2 Property Binding (One-way: Class to DOM)

```
<img [src]="imageUrl" [alt]="imageAlt">
<button [disabled]="isLoading">Submit</button>
```

7.6.3 Event Binding (One-way: DOM to Class)

```
<button (click)="onLogin()">Login</button>
<input (input)="onSearch($event)">
```

7.6.4 Two-Way Binding (ngModel)

```
<input [(ngModel)]="email" placeholder="Enter␣email">
<p>You typed: {{ email }}</p>
```

7.7 Navbar Component

```
import { Component } from '@angular/core';
import { RouterModule } from '@angular/router';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-navbar',
  standalone: true,
  imports: [RouterModule, CommonModule],
  template: `
    <nav class="navbar">
      <a routerLink="/" class="brand">The Elevated Navigator</a>
      <div class="nav-links">
        <a routerLink="/home" routerLinkActive="active">Home</a>
        <a routerLink="/dashboard" routerLinkActive="active">
           Dashboard</a>
        <a routerLink="/login">Login</a>
      </div>
    </nav>
  `,
})
export class NavbarComponent {}
```

7.8 Routing Configuration (app.routes.ts)

```
import { Routes } from '@angular/router';
import { HomeComponent } from './components/home/home.component';
import { DashboardComponent } from './components/dashboard/
   dashboard.component';
import { LoginComponent } from './components/login/login.component
   ';

export const routes: Routes = [
  { path: '',          component: HomeComponent },
  { path: 'home',      component: HomeComponent },
  { path: 'dashboard', component: DashboardComponent },
  { path: 'login',     component: LoginComponent },
  { path: '**', redirectTo: '/home' },
];
```

7.9 Root App Component (app.ts)

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { NavbarComponent } from './components/navbar/navbar.
   component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, NavbarComponent],
  template: `
    <app-navbar></app-navbar>
    <router-outlet></router-outlet>
  `,
})
export class App {}
```

7.10 Running the Angular Application

```
ng serve
# Application running at: http://localhost:4200/
```

7.11 Advantages of Angular Components

- **Reusability**: Components can be used across multiple pages
- **Encapsulation**: Each component manages its own logic and styles
- **Testability**: Components can be unit tested in isolation
- **Maintainability**: Small, focused components are easier to maintain

- **Separation of Concerns**: Template, logic, and styles are clearly separated

8. Practical No. 8: Building a Single-Page Application with Angular

8.1 Aim of the Practical

To build a Single-Page Application (SPA) using Angular with client-side routing, route guards, and navigation between views without full page reloads.

8.2 Introduction to SPA

A Single-Page Application (SPA) loads a single HTML page and dynamically updates the content as the user interacts with the app. Instead of requesting a new page from the server on every navigation, the browser loads only the necessary data and re-renders the relevant portion of the UI. Angular's `RouterModule` enables SPA behavior through client-side routing.

8.3 Key SPA Concepts in Angular

- **Router**: Manages navigation between views without page reload
- **RouterOutlet**: Placeholder where routed components are rendered
- **RouterLink**: Directive for declarative navigation in templates
- **ActivatedRoute**: Provides access to route parameters and query strings
- **Route Guard**: Controls access to routes (e.g., authentication check)
- **Lazy Loading**: Loads feature modules only when their route is visited

8.4 SPA Route Configuration (app.routes.ts)

```
import { Routes } from '@angular/router';
import { HomeComponent } from './components/home/home.component';
import { DashboardComponent } from './components/dashboard/
   dashboard.component';
import { LoginComponent } from './components/login/login.component
   ';
import { AboutComponent } from './components/about/about.component
   ';
import { authGuard } from './guards/auth.guard';

export const routes: Routes = [
  { path: '',          component: HomeComponent },
  { path: 'home',      component: HomeComponent },
  { path: 'about',     component: AboutComponent },
  { path: 'login',     component: LoginComponent },
  // Protected route     only accessible when logged in
  { path: 'dashboard', component: DashboardComponent,
    canActivate: [authGuard] },
```

```
  // Wildcard     redirect unknown paths to home
  { path: '**', redirectTo: '/home', pathMatch: 'full' },
];
```

8.5 App Configuration (app.config.ts)

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withFetch } from '@angular/common/http
   ';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(withFetch()),
  ],
};
```

8.6 Route Guard (guards/auth.guard.ts)

A route guard prevents unauthenticated users from accessing protected pages:

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';

export const authGuard: CanActivateFn = () => {
  const router = inject(Router);
  const token = localStorage.getItem('token');

  if (token) {
    return true;           // Allow navigation
  }
  router.navigate(['/login']);
  return false;            // Block navigation
};
```

8.7 Navigating Between Routes

8.7.1 Declarative Navigation (Template)

```
<!-- RouterLink for navigation without page reload -->
<a routerLink="/home" routerLinkActive="active">Home</a>
<a routerLink="/dashboard">Dashboard</a>

<!-- Navigate with parameters -->
<a [routerLink]="['/itinerary', item.id]">View Details</a>
```

8.7.2 Programmatic Navigation (Component Class)

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';

@Component({ selector: 'app-login', standalone: true, template: ''
    })
export class LoginComponent {
  constructor(private router: Router) {}

  onLoginSuccess() {
    // Navigate to dashboard after login
    this.router.navigate(['/dashboard']);
  }

  goHome() {
    this.router.navigateByUrl('/home');
  }
}
```

8.8 Reading Route Parameters

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({ selector: 'app-detail', standalone: true, template:
   '' })
export class DetailComponent implements OnInit {
  itemId = '';

  constructor(private route: ActivatedRoute) {}

  ngOnInit() {
    // Read :id from /itinerary/:id
    this.itemId = this.route.snapshot.paramMap.get('id') || '';
  }
}
```

8.9 About Component (SPA Page Example)

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-about',
  standalone: true,
  imports: [CommonModule],
  template: `
    <main class="about-page">
```

```
      <h1>About This Application</h1>
      <p>This MEAN Stack application demonstrates all 9 practicals
         :</p>
      <ul>
        <li *ngFor="let p of practicals; let i = index">
          <strong>Practical {{ i + 1 }}:</strong> {{ p }}
        </li>
      </ul>
    </main>
  `,
})
export class AboutComponent {
  practicals = [
    'Node.js, npm and Express setup',
    'Express server and static site',
    'Bootstrap responsive layouts',
    'MongoDB installation and CRUD',
    'Data modelling with Mongoose',
    'Connecting Express to MongoDB',
    'Angular components',
    'Single-Page Application with Angular',
    'Login authentication with MEAN stack',
  ];
}
```

8.10 How SPA Differs from Traditional Web Apps

- **No Full Reload**: Only the changed component re-renders, not the entire page
- **Faster Navigation**: Subsequent page transitions are near-instant
- **Better UX**: Smooth transitions without white-flash between pages
- **State Preservation**: Application state (e.g., logged-in user) persists across routes
- **API-Driven**: Backend serves JSON data; Angular renders the UI

8.11 Advantages of Angular SPA

- **Performance**: Reduced server load; only data is fetched, not full HTML pages
- **Modular**: Each route maps to an isolated component
- **Route Guards**: Fine-grained access control per route
- **Deep Linking**: URLs remain bookmarkable and shareable
- **Lazy Loading**: Large apps load faster by splitting code per route

9. Practical No. 9: Login Page Web Application Using MEAN Stack Authentication

9.1 Aim of the Practical

To construct a simple login page web application that authenticates users using the complete MEAN stack with JWT (JSON Web Token) and bcrypt password hashing.

9.2 Introduction

Authentication is the process of verifying the identity of a user. In a MEAN stack application, authentication is typically implemented using:

- **bcryptjs**: Hashes passwords before storing them in MongoDB
- **jsonwebtoken (JWT)**: Issues a signed token on successful login; the client sends this token with every subsequent request
- **Angular AuthService**: Stores the JWT in `localStorage` and exposes reactive login state
- **HTTP Interceptor**: Automatically attaches the JWT to outgoing API requests

9.3 Installation

```
# Backend
npm install bcryptjs jsonwebtoken dotenv

# Frontend (Angular)
# No extra packages needed     uses built-in HttpClient
```

9.4 Project Structure

```
mean/
|-- server/
|   |-- config/db.js
|   |-- models/User.js
|   |-- middleware/auth.js        # JWT verify middleware
|   |-- routes/authRoutes.js      # /register and /login endpoints
|-- frontend/src/app/
|   |-- services/
|   |   |-- auth.service.ts       # JWT storage and reactive state
|   |   |-- auth.interceptor.ts   # Attaches Bearer token to
   requests
|   |-- guards/auth.guard.ts      # Protects dashboard route
|   |-- components/
|       |-- login/                # Login form component
|       |-- register/             # Register form component
```

9.5 Backend: Auth Middleware (middleware/auth.js)

```
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');

const JWT_SECRET = process.env.JWT_SECRET || 'mean_secret_key';

// Verify JWT on protected routes
const verifyToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1]; // Bearer
     <token>

  if (!token) return res.status(401).json({ message: 'No token
     provided' });

  try {
    req.user = jwt.verify(token, JWT_SECRET);
    next();
  } catch {
    res.status(403).json({ message: 'Invalid or expired token' });
  }
};

const generateToken = (user) =>
  jwt.sign(
    { id: user._id, email: user.email, role: user.role },
    JWT_SECRET,
    { expiresIn: '7d' }
  );

const hashPassword   = (plain) => bcrypt.hash(plain, 10);
const comparePassword = (plain, hash) => bcrypt.compare(plain,
   hash);

module.exports = { verifyToken, generateToken, hashPassword,
   comparePassword };
```

9.6 Backend: Auth Routes (routes/authRoutes.js)

```
const express = require('express');
const router  = express.Router();
const User    = require('../models/User');
const { generateToken, hashPassword, comparePassword } = require
   ('../middleware/auth');

// POST /api/auth/register
router.post('/register', async (req, res) => {
  try {
    const { name, email, password } = req.body;
```

```
    if (await User.findOne({ email }))
      return res.status(400).json({ success: false, message: '
         Email already registered' });

    const hashed = await hashPassword(password);
    const user   = await User.create({ name, email, password:
       hashed });
    const token  = generateToken(user);

    res.status(201).json({
      success: true,
      token,
      user: { id: user._id, name: user.name, email: user.email,
         role: user.role },
    });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message })
       ;
  }
});

// POST /api/auth/login
router.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email });

    if (!user || !(await comparePassword(password, user.password))
       )
      return res.status(401).json({ success: false, message: '
         Invalid credentials' });

    const token = generateToken(user);
    res.json({
      success: true,
      token,
      user: { id: user._id, name: user.name, email: user.email,
         role: user.role },
    });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message })
       ;
  }
});

module.exports = router;
```

9.7 Frontend: Auth Service (auth.service.ts)

```
import { Injectable, signal, inject, PLATFORM_ID } from '@angular/
   core';
import { isPlatformBrowser } from '@angular/common';
import { HttpClient } from '@angular/common/http';
import { Router } from '@angular/router';
import { tap } from 'rxjs/operators';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private baseUrl   = 'http://localhost:5000/api/auth';
  private isBrowser = isPlatformBrowser(inject(PLATFORM_ID));

  currentUser = signal<any>(this.loadUser());

  constructor(private http: HttpClient, private router: Router) {}

  private loadUser() {
    if (!this.isBrowser) return null;
    try { return JSON.parse(localStorage.getItem('user') || 'null
       '); }
    catch { return null; }
  }

  get token()     { return this.isBrowser ? localStorage.getItem('
     token') : null; }
  get isLoggedIn(){ return !!this.token; }

  login(email: string, password: string) {
    return this.http.post<any>(`${this.baseUrl}/login`, { email,
       password }).pipe(
      tap(res => {
        if (res.success && this.isBrowser) {
          localStorage.setItem('token', res.token);
          localStorage.setItem('user', JSON.stringify(res.user));
          this.currentUser.set(res.user);
        }
      })
    );
  }

  register(name: string, email: string, password: string) {
    return this.http.post<any>(`${this.baseUrl}/register`, { name,
        email, password }).pipe(
      tap(res => {
        if (res.success && this.isBrowser) {
          localStorage.setItem('token', res.token);
          localStorage.setItem('user', JSON.stringify(res.user));
          this.currentUser.set(res.user);
        }
      })
```

```
    );
  }

  logout() {
    if (this.isBrowser) {
      localStorage.removeItem('token');
      localStorage.removeItem('user');
    }
    this.currentUser.set(null);
    this.router.navigate(['/login']);
  }
}
```

9.8 Frontend: HTTP Interceptor (auth.interceptor.ts)

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { AuthService } from './auth.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).token;
  if (token) {
    req = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` },
    });
  }
  return next(req);
};
```

9.9 Frontend: Login Component Template

```
<div class="login-container">
  <h1>The Elevated Navigator</h1>
  <p>Welcome back to your next adventure.</p>

  <form (ngSubmit)="onSubmit()">
    <div *ngIf="error" class="error-msg">{{ error }}</div>

    <label>Email Address</label>
    <input [(ngModel)]="email" name="email" type="email"
           placeholder="you@example.com" required>

    <label>Password</label>
    <input [(ngModel)]="password" name="password" type="password"
           placeholder="                          " required>

    <button type="submit" [disabled]="loading">
      {{ loading ? 'Signing in...' : 'Login to Dashboard' }}
    </button>
```

```
  </form>

  <p>Don't have an account?
    <a routerLink="/register">Register for early access</a>
  </p>
</div>
```

9.10 Frontend: Login Component Class

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { RouterModule, Router } from '@angular/router';
import { AuthService } from '../../services/auth.service';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [CommonModule, FormsModule, RouterModule],
  templateUrl: './login.component.html',
})
export class LoginComponent {
  email = ''; password = ''; loading = false; error = '';

  constructor(private auth: AuthService, private router: Router)
     {}

  onSubmit() {
    this.loading = true; this.error = '';
    this.auth.login(this.email, this.password).subscribe({
      next:  (res) => { this.loading = false;
                        if (res.success) this.router.navigate(['/
                           dashboard']); },
      error: (err) => { this.loading = false;
                        this.error = err?.error?.message || '
                           Invalid credentials'; },
    });
  }
}
```

9.11 Authentication Flow

1. User submits email and password on the login form
2. Angular sends a `POST /api/auth/login` request to the Express server
3. Express finds the user in MongoDB and compares the password with `bcrypt.compare()`
4. On success, Express signs a JWT with `jsonwebtoken` and returns it

5. Angular stores the JWT in `localStorage` and updates the reactive `currentUser` signal
6. The HTTP interceptor attaches `Authorization: Bearer <token>` to all future requests
7. The route guard reads the token from `localStorage` to allow or deny access to `/dashboard`

9.12 Security Best Practices

- **Never store plain-text passwords**: Always hash with bcrypt before saving
- **Use environment variables**: Keep `JWT_SECRET` in `.env`, never in source code
- **Set token expiry**: Short-lived tokens (e.g., `7d`) reduce risk if stolen
- **HTTPS in production**: Prevents token interception over the network
- **Validate on the server**: Never trust client-side auth alone; always verify the JWT on protected API routes

9.13 Advantages of JWT Authentication in MEAN Stack

- **Stateless**: Server does not store session data; scales horizontally
- **Self-contained**: Token carries user info, reducing database lookups
- **Cross-domain**: Works seamlessly with Angular calling a separate Express API
- **Secure**: bcrypt hashing makes brute-force attacks computationally expensive
- **Reactive UI**: Angular signals update the navbar and guards instantly on login/logout